Room Reservation System

Design Patterns Implementations

Michel MUSTAFOV

Edwin WEHBE

TABLE OF CONTENTS

1. Introduction

This document describes the implementation of a Room Reservation System built in Rust. The project demonstrates the practical application of 7 design patterns from the Gang of Four (GoF) catalog.

The goal is to show how design patterns solve real architectural problems: ensuring single instances, creating objects flexibly, managing hierarchies, optimizing memory, adapting interfaces, swapping algorithms, and decoupling event handling.

2. Project Architecture

The project follows a modular architecture:

Layer	Location	Responsibility
Models	src/models/	Data structures: Room, Reservation
Patterns	src/patterns/	All 7 design pattern implementations
Services	src/services/	Business logic, state management
API	src/main.rs	HTTP server, REST endpoints
Frontend	static/	Web UI: HTML, CSS, JavaScript

File structure:

```
src/patterns/
    ├── singleton.rs   → ActivityLogger, ConfigManager
    ├── factory.rs     → RoomFactory trait + concrete factories
    ├── composite.rs   → Reservable trait, Room, RoomGroup
    ├── flyweight.rs   → RoomTypeInfo, RoomTypeFlyweight
    ├── strategy.rs    → ValidationStrategy + implementations
    ├── observer.rs    → ReservationObserver, EventPublisher
    └── adapter.rs     → CalendarAdapter + implementations
```

3. Design Patterns Overview

Summary of all 7 implemented patterns:

Pattern	Category	Problem It Solves
Singleton	Creational	Ensures ONE global logger and config instance
Factory Method	Creational	Creates rooms by type without exposing instantiation logic
Composite	Structural	Treats single rooms and room groups uniformly
Flyweight	Structural	Shares room type metadata to save memory
Adapter	Structural	Integrates external calendar APIs with unified interface
Strategy	Behavioral	Swaps validation rules at runtime based on user role
Observer	Behavioral	Notifies multiple systems when reservation events occur

4. Creational Patterns

4.1 Singleton Pattern

Problem	Need exactly ONE instance of ActivityLogger and ConfigManager across the entire application
Solution	Use Rust's once_cell::Lazy with Arc<RwLock<T>> for thread-safe lazy initialization
Location	src/patterns/singleton.rs
Key Components	ActivityLogger, ConfigManager, ACTIVITY_LOG (global static), CONFIG (global static)

How it works:

- ACTIVITY_LOG is initialized on first access, then reused everywhere
- Thread-safe: multiple HTTP requests can log simultaneously via RwLock
- Stores last 500 activity entries (reservations, cancellations, errors)

4.2 Factory Method Pattern

Problem	Need to create different room types (Conference, Meeting, Training) with specific default equipment
Solution	Define RoomFactory trait. Each concrete factory creates rooms with appropriate defaults
Location	src/patterns/factory.rs
Key Components	RoomFactory (trait), ConferenceRoomFactory, MeetingRoomFactory, TrainingRoomFactory, RoomFactoryManager

Factory types and default equipment:

Factory	Default Equipment
ConferenceRoomFactory	Projector, Video Conference, Sound System
MeetingRoomFactory	Whiteboard, Phone, WiFi
TrainingRoomFactory	Projector, Whiteboard, Computers
AuditoriumFactory	Microphone, Sound System, Stage Lighting

5. Structural Patterns

5.1 Composite Pattern

Problem	Need to book entire floors or departments as easily as individual rooms
Solution	Reservable trait implemented by both Room (leaf) and RoomGroup (composite)
Location	src/patterns/composite.rs
Key Components	Reservable (trait), Room, RoomGroup, RoomGroupType (Floor/Building/Department)

Example: Call total_capacity() on a Floor → recursively sums all rooms inside.

5.2 Flyweight Pattern

Problem	100 Conference rooms each storing the same description string wastes memory
Solution	Share immutable type info via Arc pointers. All Conference rooms point to ONE description
Location	src/patterns/flyweight.rs
Key Components	RoomTypeInfo (shared data), RoomTypeFlyweight (cache manager), FlyweightRoom

5.3 Adapter Pattern

Problem	Google Calendar, Outlook, iCal have different APIs - would need multiple if/else branches
Solution	CalendarAdapter trait provides unified interface. Each adapter translates to specific API
Location	src/patterns/adapter.rs
Key Components	CalendarAdapter (trait), GoogleCalendarAdapter, OutlookAdapter, ICalAdapter

Unified adapter methods:

- sync_event(reservation) → Creates event in external calendar
- delete_event(id) → Removes event from external calendar
- check_conflicts(time) → Checks availability in external calendar

6. Behavioral Patterns

6.1 Strategy Pattern

Problem	Different users need different validation rules: standard (4h max), VIP (8h), Admin (unlimited)
Solution	ValidationStrategy trait with interchangeable implementations. Strategy selected at runtime
Location	src/patterns/strategy.rs
Key Components	ValidationStrategy (trait), StandardValidation, VipValidation, AdminValidation

Validation rules by strategy:

Strategy	Max Duration	Advance Booking	Weekends
Standard	4 hours	30 days	No
VIP	8 hours	90 days	Yes
Admin	Unlimited	Unlimited	Yes

6.2 Observer Pattern

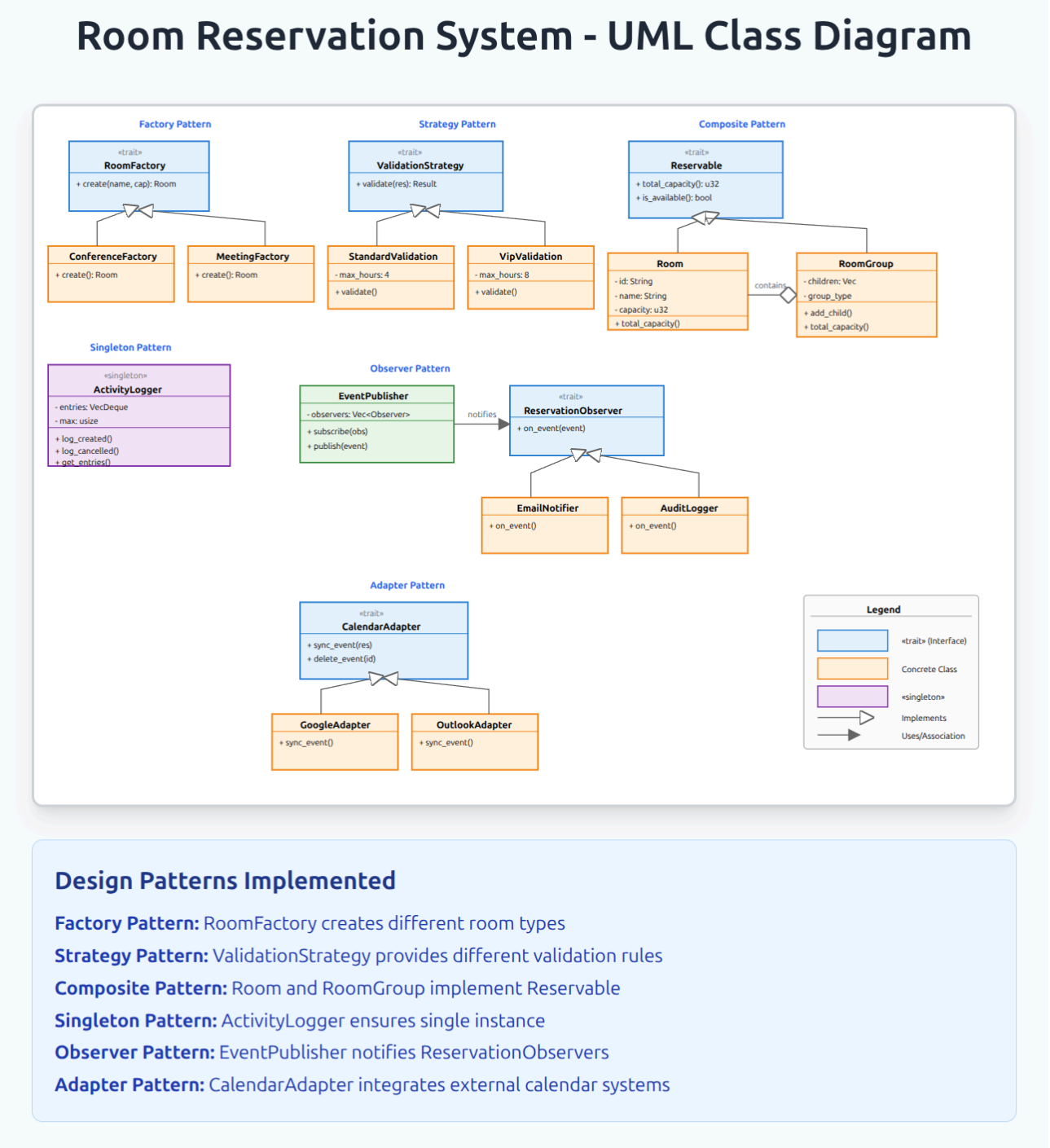
Problem	When reservation created: send email, log audit, update analytics, notify Slack - without tight coupling
Solution	EventPublisher notifies all subscribed observers. Each observer reacts independently
Location	src/patterns/observer.rs
Key Components	ReservationObserver (trait), EventPublisher, EmailNotifier, AuditLogger, SlackNotifier

Event types and observer reactions:

- ReservationCreated → Email confirmation + Audit log + Analytics tracking
- ReservationCancelled → Cancellation email + Audit log + Slack notification
- ReservationConfirmed → Reminder email + Calendar sync

7. UML Class Diagram

The following diagram shows the relationships between key classes and patterns:



Design Patterns Implemented

Factory Pattern: RoomFactory creates different room types

Strategy Pattern: ValidationStrategy provides different validation rules

Composite Pattern: Room and RoomGroup implement Reservable

Singleton Pattern: ActivityLogger ensures single instance

Observer Pattern: EventPublisher notifies ReservationObservers

Adapter Pattern: CalendarAdapter integrates external calendar systems

8. How to Run & Test

8.1 Prerequisites

- Rust 1.75+ → Install from https://rustup.rs

8.2 Quick Start

```
# 1. Extract the project
unzip room-reservation-system.zip
cd room-reservation-system

# 2. Build and run
cargo run --release

# 3. Open http://127.0.0.1:8080
```

8.3 Run Tests

```
cargo test
# Output: 23 tests passing
```

8.4 API Endpoints

Method	Endpoint	Description
GET	/api/rooms	List all rooms
POST	/api/rooms	Create room (uses Factory pattern)
GET	/api/reservations	List all reservations
POST	/api/reservations	Create reservation (uses Strategy for validation)
DELETE	/api/reservations/{id}	Cancel reservation (triggers Observer notifications)
GET	/api/logs	View activity log (from Singleton logger)